TECHNICAL DOCUMENT

System Architecture

docs/ARCHITECTURE.md

Hexagonal · Microservices · Cloud-Native

PLATFORM	STACK	GENERATED
EduTech v1.0	Java 17 · Spring Boot 3.3.5	March 07, 2026

EduTech AI Platform — System Architecture

> **Document status:** Living reference — reflects the platform as of 2026-03-07.
> All service completion documents (COMPLETION.md) are the authoritative frozen record for each service. This document provides cross-cutting architectural context.

Table of Contents

1. System Overview
2. High-Level Architecture Diagram
3. Hexagonal Architecture Pattern
4. Service Inventory
5. Data Architecture
6. Messaging Architecture
7. Security Architecture
8. Reactive Services
9. Observability Stack
10. ArchUnit Enforcement

1. System Overview

EduTech AI Platform is an enterprise-grade, AI-first educational technology platform designed to digitize and intelligently automate coaching center operations end-to-end. It serves three primary user personas:

PERSONA	PRIMARY ACTIONS
Coaching Center Admins / Teachers	Manage centers, batches, schedules, content, and attendance
Parents	Track children's performance, view reports, record payments
Students	Take adaptive assessments, receive psychometric profiling, view results

The platform is delivered as a React Native Android APK (with iOS and web targets sharing the same codebase) and exposes a REST API surface behind a single edge gateway.

Architectural characteristics:

- **Cloud-native:** Containerized on Kubernetes 1.30 with Helm 3 + ArgoCD GitOps; distroless Java 21 images with multi-stage Docker builds.
- **Microservices:** Seven independently deployable services with strict bounded-context isolation. Zero shared databases.
- **AI-powered:** All AI requests are routed through a dedicated reactive AI gateway service that integrates Anthropic Claude (primary LLM), OpenAI Embeddings, and a Python FastAPI ML sidecar for psychometric inference.
- **Hexagonal architecture (Ports and Adapters):** Every service enforces the same four-layer package structure via ArchUnit rules that run on every build.
- **Zero-trust security:** JWT RS256 asymmetric tokens validated at the edge; downstream services trust forwarded headers. Istio mTLS STRICT between all pods.

Non-functional targets:

ATTRIBUTE	TARGET
Availability SLA	99.99% (< 52 minutes downtime per year)
API latency (critical path)	P99 < 100 ms
Password hashing	Argon2id — 64 MB memory, 3 iterations, parallelism 2
Token security	JWT RS256; 15-min access token; 7-day single-use rotating refresh token
Test coverage	>= 85% line, >= 80% branch per service
Throughput baseline	1,000 RPS sustained, P99 < 200 ms (Gatling validated)

---

2. High-Level Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                React Native Mobile App (APK / IPA)                       │
│           Expo SDK 51 · Expo Router · MMKV · expo-secure-store           │
└──────────────────────────────┬───────────────────────────────────────────┘
                               │ HTTPS (TLS 1.3)
                               │
┌──────────────────────────────▼───────────────────────────────────────────┐
│                       api-gateway  :8090                                 │
│                                                                          │
│  GlobalFilter (JwtAuthenticationFilter, order -1)                        │
│    - RS256 validation via JJWT 0.12.x + RSA public key PEM               │
│    - Forwards X-User-Id · X-User-Role · X-User-Center-Id downstream      │
│  RequestIdFilter (order -2) — injects X-Request-ID UUID                  │
│  RequestRateLimiter (default-filter) — Redis token bucket                │
│    redis-rate-limiter.replenish-rate / burst-capacity                    │
│  Resilience4j CircuitBreaker per route (e.g. auth-svc instance)          │
│  OpenAPI aggregation (springdoc WebFlux UI)                              │
│                                                                          │
│  Spring Cloud Gateway (WebFlux / Reactor Netty)                          │
└──┬────────┬────────┬────────┬────────┬────────────┬──────────────────────┘
   │        │        │        │        │            │
   │ /auth  |/parents|/centers |/assess- | /psych/**  | /ai/**
   |  /**   |  /**   |  /**   | ments/**|            |
   │        │        │        │        │            │
┌──▼──┐  ┌──▼───┐  ┌──▼────┐  ┌──▼────┐  ┌──▼──────┐  ┌──▼──────────┐
|auth |  |parent|  |center |  |assess |  |psych    |  |ai-gateway   |
|-svc |  |-svc  |  |-svc   |  |-svc   |  |-svc     |  |-svc         |
|:8081|  |:8082 |  |:8083  |  |:8084  |  |:8085    |  |:8086        |
|     |  |      |  |       |  |       |  |         |  |             |
|RBAC |  |Child |  |Center |  |Exams  |  |Big Five |  |LLM routing  |
|Auth |  |links |  |Batches|  |CAT    |  |RIASEC   |  |Anthropic API |
|OTP  |  |Fees  |  |Sched. |  |Grading |  |Career   |  |OpenAI Embed. |
|JWT  |  |Notif.|  |Content|  |Proctor.|  |Mapping  |  |Redis rate lim.|
|Bio- |  |Prefs.|  |Attend.|  |IRT    |  |Session  |  |WebFlux/Mono |
|metr.|  |      |  |Fees   |  |Submiss.|  |Tracking |  |             |
└─────┘  └──────┘  └───────┘  └───────┘  └─────────┘  └─────────────┘
   │        │        │         │        |   WebClient    |  WebClient
   │        │        │         │        |   + CB         |
   │        │        │         │        ▼                ▼
   │        │        │         │   ┌──────────┐   ┌──────────────┐
   │        │        │         │   | psych-ai-svc|  | Anthropic API |
   │        │        │         │   | :8095    |   | (Claude)     |
   │        │        │         │   | Python   |   ├──────────────┤
   │        │        │         │   | FastAPI  |   | OpenAI API   |
   │        │        │         │   | scikit-learn|  | (Embeddings) |
   │        │        │         │   | RandomForest|  └──────────────┘
   │        │        │         │   | SHAP     |
   │        │        │         │   └──────────┘
```

```
│        │        │          │
└────────┴────────┴──────────┴──────────────────────────────────┐
                                                                 │
┌──────────────────────────────────────────────────┐             │
│              Apache Kafka 3.6 (KRaft, no ZooKeeper)  │ ◄──────────┘
│  Topics: auth-events · center-events · parent-events │
│          assess-events · psych-events                │
│          ai-gateway-events · audit-immutable         │
└──────────────────────────────────────────────────┘
                  │
┌─────────────────▼────────────────────────────────┐
│         PostgreSQL 16 (+ pgvector + TimescaleDB)      │
│  auth_schema · parent_schema · center_schema          │
│  assess_schema · psych_schema                         │
│  (one schema per service, single instance in local dev│
│   separate instances in production)                   │
└──────────────────────────────────────────────────┘
                  │
┌─────────────────▼────────────────────────────────┐
│                 Redis 7 Cluster                       │
│  auth-svc: refresh tokens, OTP, behavioral scores    │
│  api-gateway: rate limiter (token bucket)             │
│  ai-gateway-svc: per-requester rate limit counters   │
└──────────────────────────────────────────────────┘
```

Public paths on api-gateway (no JWT required):

`/api/v1/auth/**`, `/actuator/**`, `/swagger-ui/**`, `/v3/api-docs/**`, `/webjars/**`

---

3. Hexagonal Architecture Pattern

Every service in the platform (except api-gateway, which is configuration-driven) uses the same four-layer hexagonal structure. The dependency rule is absolute and enforced by ArchUnit at build time.

Layer Diagram

```
┌──────────────────────────────────────────────────────────────┐
│  api/                                                        │
│    REST controllers (@RestController)                        │
│    GlobalExceptionHandler (@RestControllerAdvice — RFC 7807) │
│    MapStruct mappers (compile-time, zero reflection)         │
│    Depends on: application only                              │
├──────────────────────────────────────────────────────────────┤
│  application/                                                │
│    Use case interfaces (port/in/)                            │
│    Application services (implements use cases)               │
│    DTOs — Java Records (immutable by language spec)          │
```

```
│    Domain exceptions (all extend abstract *Exception base)          │
│    @ConfigurationProperties beans                                   │
│    Depends on: domain only (+ SLF4J, Jackson where needed)          │
├─────────────────────────────────────────────────────────────────────┤
│  domain/                                                            │
│    JPA entities — no public setters, state via named domain methods │
│    Enums for all state machines                                     │
│    Domain events — Java Records (immutable, published to Kafka)     │
│    Port interfaces (port/in/ use cases, port/out/ repositories)     │
│    @Version on all mutable entities (optimistic locking)            │
│    ZERO Spring dependencies. ZERO infrastructure imports.           │
├─────────────────────────────────────────────────────────────────────┤
│  infrastructure/                                                    │
│    Persistence: Spring Data JPA (package-private *Repository        │
│      interfaces, public *PersistenceAdapter implements domain port) │
│    Security: JwtTokenValidator, JwtAuthenticationFilter,            │
│      SecurityConfig                                                 │
│    Messaging: *EventKafkaAdapter (implements domain event publisher)│
│      *EventConsumer (@KafkaListener — phase-stub in most services)  │
│    External: WebClient adapters to Python sidecar / third-party APIs│
│    Redis: RedisTemplate / ReactiveRedisTemplate adapters            │
│    Depends on: domain and application                               │
│    Does NOT depend on: api                                          │
└─────────────────────────────────────────────────────────────────────┘
```

Package Naming Convention

```
com.edutech.{service}
├── {Service}Application.java         @SpringBootApplication @ConfigurationPropertiesScan
├── domain/
│   ├── model/                        JPA entities, enums, value objects
│   ├── event/                        Immutable domain event records
│   └── port/
│       ├── in/                       Use case interfaces
│       └── out/                      Repository + publisher port interfaces
├── application/
│   ├── config/                       @ConfigurationProperties records
│   ├── dto/                          Java Records for request/response
│   ├── exception/                    Typed domain exceptions
│   └── service/                      Use case implementations
├── infrastructure/
│   ├── config/                       Kafka topic properties, etc.
│   ├── security/                     JWT filter, SecurityConfig
│   ├── persistence/                  JPA adapters
│   ├── messaging/                    Kafka producers and consumers
│   ├── redis/                        Redis adapters (auth-svc)
│   └── external/                     WebClient adapters (psych-svc, ai-gateway-svc)
```

```
└── api/
    ├── GlobalExceptionHandler.java
    ├── *Controller.java
    └── mapper/                        MapStruct interfaces (auth-svc only)
```

Design Invariants (Platform-Wide)

INVARIANT	IMPLEMENTATION
No Lombok	Manual constructors and accessors everywhere
No field injection	Constructor injection only, without exception
No hardcoded values	All config flows through `${ENV_VAR}` in `application.yml`
No public setters on entities	State changes via named domain methods with guards
No physical DELETE	Soft delete via `deleted_at` timestamp; all queries filter `deleted_at IS NULL`
Immutable DTOs	Java Records only
Immutable domain events	Java Records only
Optimistic locking	`@Version Long version` on all mutable JPA entities
Static factory pattern	`Entity.create(...)` sets `id = UUID.randomUUID()` before persist
Best-effort Kafka	Publish failures are logged at WARN; DB transaction is never rolled back
RFC 7807 errors	`ProblemDetail.forStatusAndDetail(status, detail)` with typed `https://edutech.com/problems/{type}` URIs
Virtual threads	`spring.threads.virtual.enabled: true` in all Spring MVC services

4. Service Inventory

SERVICE	MAVEN ARTIFACT	HTTP PORT	GRPC PORT	DB SCHEMA	KEY AGGREGATES / RESPONSIBILITIES
auth-svc	`com.edutech:auth-svc`	8081	9091	`auth_schema`	User, Role, UserStatus; JWT RS256 issuance; Argon2id hashing; OTP (TOTP RFC 6238); hCaptcha; behavioral biometrics
parent-svc	`com.edutech:parent-svc`	8082	9092	`parent_schema`	ParentProfile, StudentLink, FeePayment, NotificationPreference; child

SERVICE	MAVEN ARTIFACT	HTTP PORT	GRPC PORT	DB SCHEMA	KEY AGGREGATES / RESPONSIBILITIES
					performance dashboard; consent management
center-svc	`com.edutech:center-svc`	8083	9093	`center_schema`	CoachingCenter, Batch, Teacher, Schedule, FeeStructure, Attendance, ContentItem; multi-tenant ops backbone
assess-svc	`com.edutech:assess-svc`	8084	9094	`assess_schema`	Exam, Question (IRT params + embedding), ExamEnrollment, Submission, SubmissionAnswer (immutable), Grade; CAT engine; WebSocket/STOMP stub
psych-svc	`com.edutech:psych-svc`	8085	9095	`psych_schema`	PsychProfile (Big Five OCEAN + RIASEC), TraitDimension (reference), SessionHistory, CareerMapping; WebClient to Python psych-ai-svc sidecar
ai-gateway-svc	`com.edutech:ai-gateway-svc`	8086	—	—	LLM routing (Anthropic Claude, OpenAI, Ollama fallback); embedding generation; career prediction proxy; Redis rate limiting; fully reactive (WebFlux)
api-gateway	`com.edutech:api-gateway`	8090	—	—	Spring Cloud Gateway; JWT enforcement (GlobalFilter); RequestId injection; Redis token-bucket rate limiting; OpenAPI aggregation

Python sidecar (external process, not in this repo):

SIDECAR	PORT	FRAMEWORK	RESPONSIBILITY
psych-ai-svc	8095	Python FastAPI	scikit-learn RandomForest + SHAP for career prediction; trait embedding generation

5. Data Architecture

Database Engine

PostgreSQL 16 with two extensions:

EXTENSION	PURPOSE
`pgvector`	Vector similarity search for question embeddings (assess-svc) and psychometric trait embeddings (psych-svc). Columns stored as `TEXT` (JSON float array) in Phase 1; upgrade path to native `vector(1536)` with HNSW index documented in each service's COMPLETION.md.
`TimescaleDB`	Time-series hypertables for student performance metrics in assess-svc (`student_performance_ts`, partitioned by week).

Schema Isolation

Each service owns exactly one PostgreSQL schema. No service queries another service's schema. Cross-service data is referenced by UUID only.

```
Single PostgreSQL instance (local dev) / separate instances (production):

  auth_schema      ← auth-svc exclusive
  parent_schema    ← parent-svc exclusive
  center_schema    ← center-svc exclusive
  assess_schema    ← assess-svc exclusive
  psych_schema     ← psych-svc exclusive
```

Migration Strategy

Flyway manages all schema evolution. Conventions:

- Migration scripts: `V1__init.sql`, `V2__<description>.sql` ... `VN__<description>.sql`
- `baseline-on-migrate: false` — all schemas created fresh from migrations
- No destructive `DROP` in production migrations — additive changes only
- Reference data seeded in dedicated migration scripts (e.g., psych-svc V2 seeds 11 `trait_dimensions` rows)

Migration counts per service at freeze:

SERVICE	FLYWAY SCRIPTS
auth-svc	V1 — init
center-svc	V1 — init
parent-svc	V1 — init
assess-svc	V1–V7 (7 scripts)
psych-svc	V1–V5 (5 scripts, V2 seeds reference data)

Soft Delete Pattern

No service issues physical `DELETE` statements against mutable entity tables. All deletes set `deleted_at = NOW()`. All JPQL/HQL queries that touch mutable entities include the predicate `deleted_at IS NULL`.

```
-- Example: psych_profiles
SELECT p FROM psych_profiles p
WHERE p.center_id = :centerId
  AND p.deleted_at IS NULL
```

Exceptions: Reference/lookup tables with no lifecycle (e.g., `trait_dimensions`) do not have a `deleted_at` column — they are permanent master data altered only via DB migration.

Optimistic Locking

Every mutable JPA entity carries `@Version Long version`. Hibernate increments this on every UPDATE. A stale write (two concurrent writers on the same row) throws `OptimisticLockingFailureException`, mapped by `GlobalExceptionHandler` to HTTP 409.

Immutable Records

Certain entities are insert-only and must never be updated after creation:

ENTITY	SERVICE	IMMUTABILITY MECHANISM
`SubmissionAnswer`	assess-svc	All columns `@Column(updatable=false)`; no `@Version`; no `updatedAt`; no `deletedAt`
OTP records	auth-svc	Redis TTL-backed; DB record `@Column(updatable=false)` on all fields
Domain events (Kafka)	all	Java Records; published best-effort, never mutated
Audit log entries	all	Written to `audit-immutable` Kafka topic; retention = forever; no consumer-group deletion

Index Strategy Summary

PATTERN	IMPLEMENTATION
B-tree partial index	`WHERE deleted_at IS NULL` on all frequently-queried FK columns
BRIN index	`created_at` on append-heavy tables (session_histories, career_mappings, submissions)
GIN index	JSONB columns where applicable
UNIQUE partial index	Business-key uniqueness enforced at DB level (e.g., `UNIQUE(student_id, center_id)` on psych_profiles)

PATTERN	IMPLEMENTATION
HNSW index (Phase 2)	Vector similarity on embedding columns after pgvector activation

6. Messaging Architecture

Broker

Apache Kafka 3.6 running in **KRaft mode** (no ZooKeeper). KRaft eliminates the operational overhead of a separate ZooKeeper ensemble and provides faster controller failover.

Topic Registry

TOPIC	PARTITIONS	PRODUCER(S)	CONSUMER(S)
`auth-events`	12	auth-svc	audit-svc, notify-svc (future)
`center-events`	12	center-svc	assess-svc, parent-svc, audit-svc
`parent-events`	12	parent-svc	audit-svc, notify-svc (future)
`assess-events`	24	assess-svc	psych-svc (grade-triggered sessions), audit-svc
`psych-events`	6	psych-svc	parent-svc (future), center-svc (future), audit-svc
`ai-gateway-events`	12	ai-gateway-svc	audit-svc
`audit-immutable`	12	all services	audit-svc (read-only; retention = forever)

Event Schema

All domain events are Java `record` types defined in `domain/event/`. They are serialized to JSON via `JsonSerializer` (Spring Kafka). No Avro or schema registry in Phase 1; schema evolution is managed via additive fields and `@JsonIgnoreProperties(ignoreUnknown=true)` on consumers.

Publish Semantics

All services use **best-effort publish**: the Kafka `KafkaTemplate.send()` call is made after the DB transaction commits. If the Kafka publish fails:
- The exception is caught and logged at `WARN` level.
- The DB transaction is **not** rolled back.
- The event is silently dropped.

This is a deliberate trade-off: database consistency is authoritative; the event bus is a convenience layer. Phase 2 will introduce an outbox pattern for guaranteed delivery on critical events.

Consumer Pattern

Each service declares a Kafka consumer group ID via `${SERVICE_KAFKA_CONSUMER_GROUP}`. All consumers use `auto-offset-reset: earliest`. Phase 1 consumers primarily log received payloads; business logic reactions are Phase 2 hooks.

---

7. Security Architecture

Token Lifecycle

```
[User login with Argon2id password + hCaptcha]
        |
        ▼
  auth-svc issues:
  ┌──────────────────────────────────────────────────┐
  | Access Token (JWT RS256)                         |
  |   TTL: 15 minutes                                |
  |   Claims: sub (userId), email, role, centerId    |
  |   Signed with: RSA private key (never leaves     |
  |                auth-svc)                         |
  ├──────────────────────────────────────────────────┤
  | Refresh Token (opaque UUID)                      |
  |   TTL: 7 days                                    |
  |   Storage: Redis  key=rt:{tokenId}               |
  |   Bound to: device fingerprint hash              |
  |   Single-use: consumed on rotation               |
  └──────────────────────────────────────────────────┘
        |
        ▼
  Client sends Bearer token to api-gateway
        |
        ▼
  api-gateway JwtAuthenticationFilter (GlobalFilter order -1):
    - Loads RSA public key from PEM file at startup
    - JJWT 0.12.x: Jwts.parser().verifyWith(publicKey).requireIssuer(issuer)
    - On valid token: injects X-User-Id, X-User-Role, X-User-Center-Id headers
    - On invalid token: returns 401 ProblemDetail before routing
        |
        ▼
  Downstream services trust forwarded headers:
    - No re-validation of JWT signature
    - Extract AuthPrincipal from X-User-Id / X-User-Role / X-User-Center-Id
    - @PreAuthorize / manual role checks in application services
```

Password Hashing

```
Argon2id configuration (OWASP 2024 recommended):
  memoryCost : 65536 (64 MB)
  iterations : 3
  parallelism: 2
  saltLength : 16 bytes
  hashLength : 32 bytes
```

Refresh Token Security

- **Single-use rotation:** Each refresh token is deleted from Redis on use; a new token is issued. If a token arrives that has already been consumed, all sessions for that user are immediately revoked.
- **Device fingerprint binding:** The fingerprint is a SHA-256 hash of `user-agent + IP subnet + device-id`. A fingerprint mismatch during rotation triggers a full session purge.
- **Instant revocation:** Redis-backed storage means a token can be deleted server-side without waiting for expiry. Logout (single device or all devices) deletes the Redis key(s) immediately.

hCaptcha Bot Protection

Registration and login endpoints require hCaptcha Enterprise token verification before processing. The `HCaptchaRestAdapter` (auth-svc) calls the hCaptcha verification URL with the site key and user token. Circuit breaker protects against hCaptcha API outages; on CB open, the request is rejected (fail-closed for security).

OTP (One-Time Password)

- 6-digit TOTP per RFC 6238
- Redis TTL: 300 seconds (5 minutes)
- Max 3 attempts; counter stored as a separate Redis key; exceeded attempts lock the OTP attempt
- Delivery: SMTP email (AWS SES / SendGrid) or Twilio SMS

Downstream Service Authorization

Each service implements its own authorization model in the application service layer. Authorization is checked before any repository call. Common pattern:

```
AuthPrincipal principal = extractFromHeaders(request)
if (!principal.belongsToCenter(request.centerId())) {
    throw new *AccessDeniedException(...)
}
```

The `AuthPrincipal` record carries: `userId`, `email`, `role`, `centerId` (nullable), `deviceFP`.

Zero Trust Infrastructure

CONTROL	MECHANISM
Service-to-service mTLS	Istio PeerAuthentication STRICT mode; no plain HTTP in cluster

CONTROL	MECHANISM
Secrets management	HashiCorp Vault; dynamic DB credentials with 15-minute lease
Container scanning	Trivy in CI pipeline + Falco runtime threat detection
Network policy	Default-deny-all ingress/egress; explicit allow-list per service pair
Dependency audit	OWASP dependency-check + Dependabot auto-PRs
SBOM	CycloneDX Maven plugin; attested with sigstore (Stage 4+)

8. Reactive Services

The platform uses two distinct concurrency models based on service characteristics:

Spring WebFlux (Fully Reactive)

ai-gateway-svc and **api-gateway** are fully reactive. No blocking I/O anywhere.

COMPONENT	IMPLEMENTATION
HTTP server	Reactor Netty (embedded, WebFlux)
Security filter	`JwtAuthenticationWebFilter implements WebFilter` (not `OncePerRequestFilter`)
Security config	`@EnableWebFluxSecurity`, `ServerHttpSecurity`
Service layer	Returns `Mono<T>` / `Flux<T>` throughout
Redis	`ReactiveRedisTemplate` — non-blocking INCR/EXPIRE for rate limiting
External HTTP	`WebClient` (non-blocking)
Error handling	`GlobalExceptionHandler` returns `Mono<ResponseEntity<ProblemDetail>>`

ai-gateway-svc reactive service pattern:

```
rateLimitPort.checkAndIncrement()          // Mono<Boolean>
  .flatMap(allowed -> {
      if (!allowed) throw RateLimitExceededException
      return llmClient.complete(request)   // Mono<CompletionResponse>
  })
  .doOnSuccess(r -> { if (r != null) eventPublisher.publish(...) })
  .doOnError(e -> log.error(...))
  .onErrorMap(e -> !(e instanceof AiGatewayException), AiProviderException::new)
```

Spring MVC + Virtual Threads (Java 21)

All other services (**auth-svc**, **parent-svc**, **center-svc**, **assess-svc**, **psych-svc**) use Spring MVC with Java 21 virtual threads enabled via:

```
spring:
  threads:
    virtual:
      enabled: true
```

Virtual threads provide near-WebFlux throughput with blocking-style code, eliminating the complexity of reactive programming for services that do not require it. `WebClient.block()` in psych-svc (for the AI sidecar call) is safe under virtual threads because each virtual thread blocks on its carrier thread without pinning an OS thread.

---

9. Observability Stack

Signals

SIGNAL	TECHNOLOGY	DETAILS
Traces	OpenTelemetry SDK → OTLP exporter → Grafana Tempo	`micrometer-tracing-bridge-otel`; auto-instrumentation of all Spring Boot services; sampling probability configurable via `${OTEL_SAMPLING_PROBABILITY}`
Metrics	Micrometer → Prometheus → Grafana Mimir	90-day retention; tagged with `application` and `environment` labels per service
Logs	Logback (structured JSON) → Loki	MDC carries `traceId`, `spanId`, `userId` on every log line; correlated to Tempo traces
Profiles	Grafana Pyroscope (Stage 4+)	Continuous profiling for CPU hot-path analysis

Actuator Endpoints

All services expose the following via Spring Boot Actuator (controlled by `${ACTUATOR_ENDPOINTS}`):

```
/actuator/health     — liveness + readiness probes (Kubernetes)
/actuator/info       — service version and build metadata
/actuator/metrics    — Micrometer metric registry
/actuator/prometheus — Prometheus scrape endpoint
```

Alerting

- **P1 alerts:** PagerDuty escalation (SLO burn rate > 5x for 1 hour)
- **P2/P3 alerts:** Slack notification channel

- Alertmanager manages routing rules

Distributed Tracing Context

The `RequestIdFilter` in api-gateway injects `X-Request-ID` (UUID) into every inbound request. Downstream services pick up the trace context via OpenTelemetry auto-instrumentation propagated through HTTP headers (`traceparent`, `tracestate`).

---

10. ArchUnit Enforcement

Every service contains an `ArchitectureRulesTest.java` in `src/test/java/.../architecture/`. These tests run as part of the standard `mvn test` lifecycle and will **fail the build** if any layer violation is introduced.

Rules Applied to All Hexagonal Services

RULE	WHAT IT PREVENTS
`domain` must not depend on `infrastructure` or `api`	Spring/JPA/Kafka imports in domain entities or events
`application` must not depend on `infrastructure` or `api`	Direct JPA repository calls in application services; controller DTO leakage
`infrastructure` must not depend on `api`	Persistence adapters importing REST controller classes
`api` must not depend on `infrastructure`	Controllers calling JPA repositories directly
Services (`.*Service`) must reside in `application.service`	Prevents service logic from migrating into infrastructure or api packages

Rules Applied to api-gateway (Configuration-Driven)

Since api-gateway has no hexagonal layers, its ArchUnit rules guard a different set of invariants:

RULE	WHAT IT PREVENTS
`@ConfigurationProperties` classes in `config` package	Configuration scattered across arbitrary packages
`GlobalFilter` implementations in `security` or `filter` package	Filter logic mixed into business packages
No JPA `@Entity` anywhere in the gateway	ORM bleeding into what must remain a stateless router
No `@Service` annotation — only `@Component` and `@Configuration`	Misleading layered-architecture artifacts in a non-layered service
`filter` package classes must not access `security` package internals	One-way dependency from filter to security only

Test Results at Freeze (2026-03-07)

SERVICE	ARCHUNIT RULES	TOTAL TESTS	RESULT
auth-svc	5/5 PASS	10/10	BUILD SUCCESS
center-svc	5/5 PASS	11/11	BUILD SUCCESS
parent-svc	5/5 PASS	11/11	BUILD SUCCESS
assess-svc	5/5 PASS	11/11	BUILD SUCCESS
psych-svc	5/5 PASS	12/12	BUILD SUCCESS
ai-gateway-svc	5/5 PASS	10/10	BUILD SUCCESS
api-gateway	5/5 PASS	10/10	BUILD SUCCESS

EduTech AI Platform — Architecture Document v1.0 — 2026-03-07

Stack: Java 21 · Spring Boot 3.3 · PostgreSQL 16 · Kafka 3.6 · Redis 7 · React Native 0.74